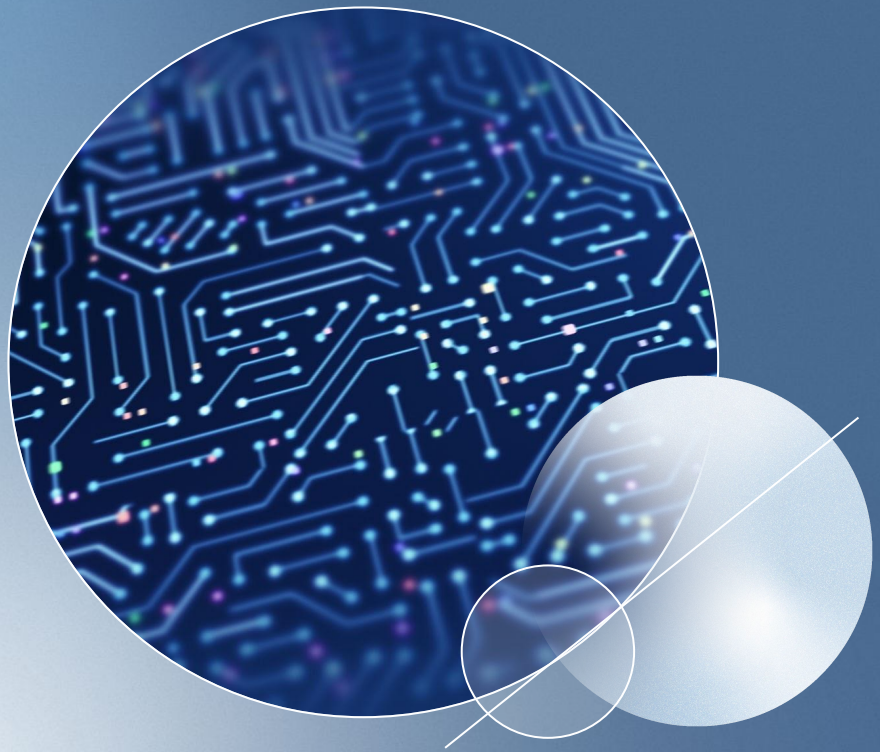
PANDAS DATAFRAME

MODULE 4

Learning Objectives



By the end of this week, students should be able to:

1. Use Pandas DataFrame for data query, heatmap generation, correlation analysis, and plotting

Pandas Basics

- High-level data manipulation tool
- Built on top of Numpy
- DataFrame is Pandas key data structure

```
>>> furniture
   Unnamed: 0        Product      Brand     Cost
0           1           Sofa      Sam's  35000.0
1           2            Bed    Darien'  50000.0
2           3     Nightstand   Stephen's  11000.0
3           4   Coffee table      Sam's  19000.0
4           5       Wall art       Doy's   7777.0
```

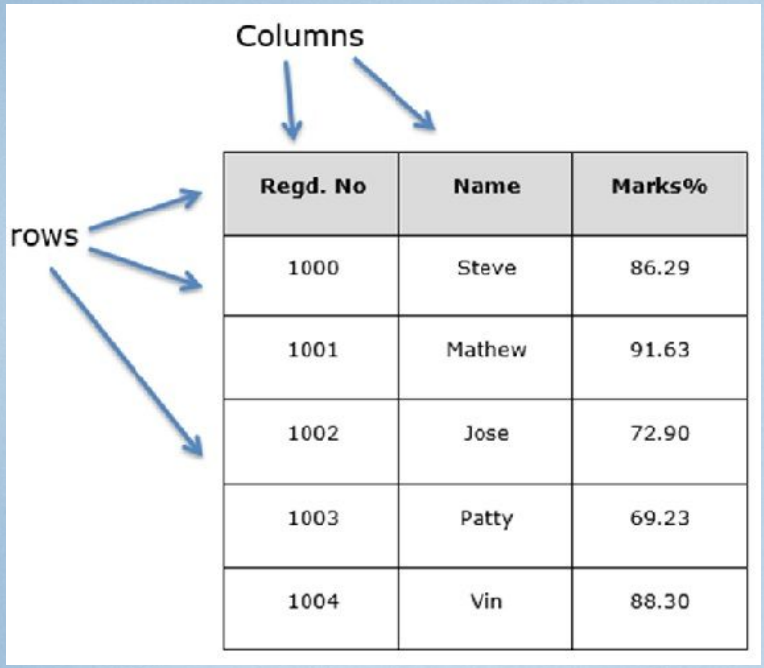
Pandas DataFrame stores data in a row-and-column structure

Pandas is a Python package that provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive.

It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python.

Additionally, it has the broader goal of becoming the most powerful and flexible open source data analysis/manipulation tool available in any language. It is already well on its way towards this goal.

In Pandas, the key data structure is called DataFrame. A DataFrame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns.



Pandas DataFrames have a row-and-column structure.

Features of DataFrame

- Potentially columns are of different types (different from Numpy arrays)
- Size – mutable
  - Sizes being mutable refers that elements can be appended or deleted/pop-ed from a DataFrame.
  - On the contrary, a series is size immutable, which means once a series object is created operations such as appending/deleting, which would change the size of the object are not allowed.
- Labeled axes (rows and columns)
- Can perform arithmetic operations on rows and columns

- Columns can be of different types
- Size - mutable
- Axes - labeled (rows and columns)
- Arithmetic operations on rows and columns

Ways to create a Pandas DataFrame

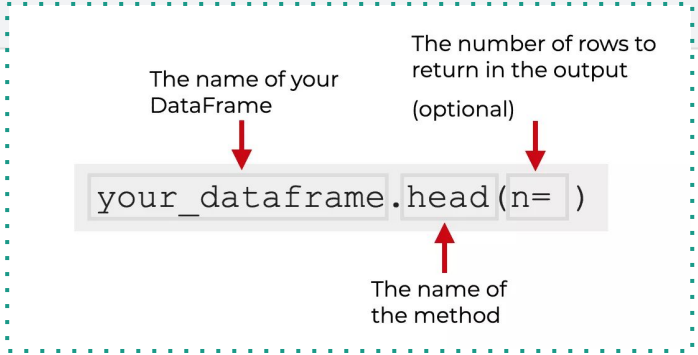
```
import pandas as pd
data = [['Alex',10],['Bob',12],['Clarke',13]]
df = pd.DataFrame(data,columns=['Name','Age'])
print df
```

```
import pandas as pd
data = {'Name':['Tom', 'Jack', 'Steve', 'Ricky'],'Age':[28,34,29,42]}
df = pd.DataFrame(data)
print df
```

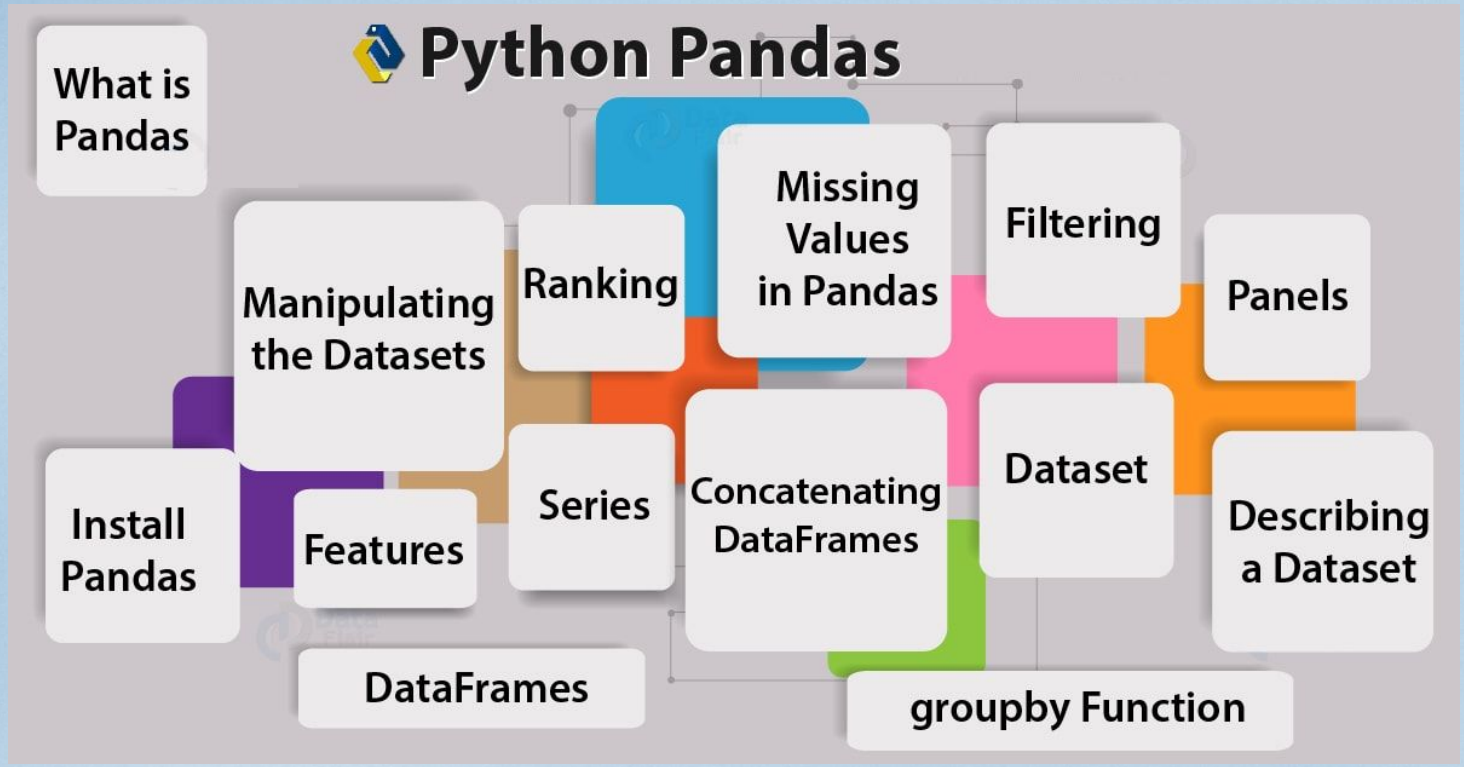
Create a Pandas DataFrame by reading CSV

```
import pandas as pd

df = pd.read_csv('https://raw.githubusercontent.com/huizhangky/CSE590/main/hw/hw2/subject_registry.csv')

print(df.head(10))
```

	Site	SubjectID	ISA	Actual_Visit	Data_collection_started
0	100	11609	NaN	SF	3-Aug-21
1	100	12140	NP03	Complete	10-Aug-21
2	100	12120	BP03	Complete	11-Aug-21
3	100	11806	BP03	Complete	12-Aug-21
4	100	11653	BP03	ED	12-Aug-21
5	100	11530	BP03	Complete	25-Aug-21
6	102	12629	BP03	ED	14-Oct-21
7	102	12902	BP03	V3	3-Dec-21
8	103	12452	NP03	ED	23-Aug-21
9	103	12638	BP03	ED	7-Oct-21



The Pandas head() method is very simple.
The head() method returns the first few rows of a Pandas dataframe


Python Pandas
What is Pandas
Manipulating the Datasets
Ranking
Missing Values in Pandas
Filtering
Panels
Install Pandas
Features
Series
Concatenating DataFrames
Dataset
Describing a Dataset
DataFrames
groupby Function

Sorting DataFrames

You may use df.sort_values in order to sort Pandas DataFrame.

- E.g.,
  - A column in an ascending order
  - A column in a descending order

You can also sort on more than 1 column

```
df.sort_values('Data_collection_started',ascending=True)
```

[7]:

	Site	SubjectID	ISA	Actual_Visit	Data_collection_started
160	122	11232	NaN	SF	2021-05-17
118	111	11915	NP03	ED	2021-05-17
163	122	11720	BP03	Complete	2021-05-18
120	111	11911	BP03	ED	2021-05-19
162	122	11696	BP03	Complete	2021-05-19
...	...	...	...	...	...
286	147	12972	NaN	V2	2021-12-23
218	122	12970	NaN	V1	2021-12-23
219	122	12881	NaN	V2	2021-12-23
92	109	12928	NaN	V2	2021-12-27
93	109	12773	NaN	V2	2021-12-27

289 rows × 5 columns

Filtering DataFrames

Pandas dataframe.filter() function is used to subset rows or columns of dataframe with user defined conditions.

```
BP03_df = df[df['ISA'] == "BP03"]
```

```
BP03_df
```

	Site	SubjectID	ISA	Actual_Visit	Data_collection_started
2	100	12120	BP03	Complete	2021-08-11
3	100	11806	BP03	Complete	2021-08-12
4	100	11653	BP03	ED	2021-08-12
5	100	11530	BP03	Complete	2021-08-25
6	102	12629	BP03	ED	2021-10-14
...	...	...	...	...	...
277	147	12584	BP03	Complete	2021-09-27
278	147	12595	BP03	Complete	2021-09-23
279	147	12572	BP03	ED	2021-10-07
282	147	12654	BP03	Complete	2021-10-21
285	147	12821	BP03	V3	2021-11-17

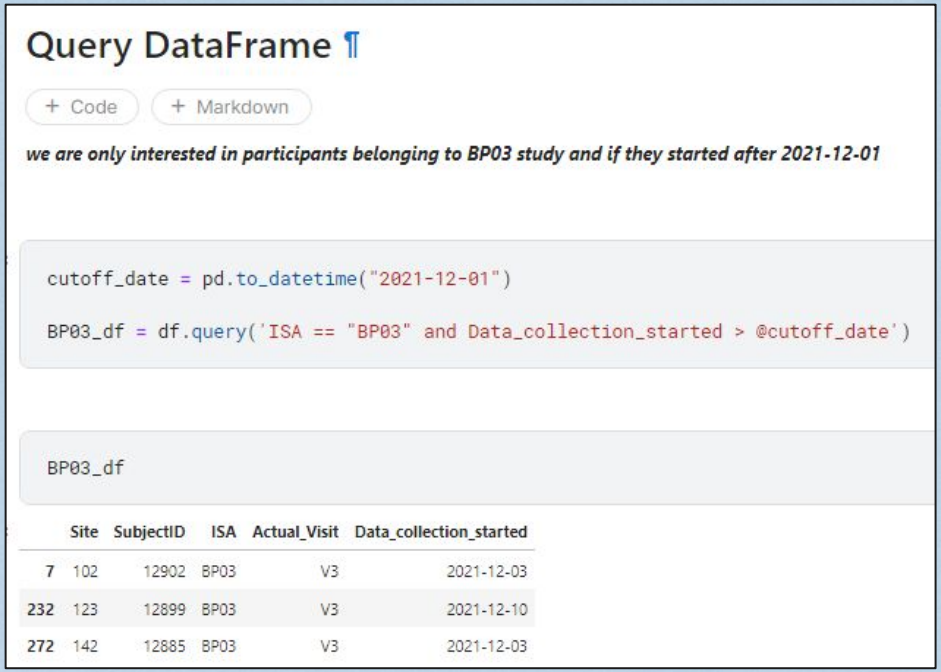
132 rows × 5 columns

Querying with SQL

To query DataFrame rows based on a condition applied on columns, you can use pandas.DataFrame.query() method.

The syntax of Pandas query is mostly straightforward, very much like a SQL query.

By default, query() function returns a DataFrame containing the filtered rows. You can also pass inplace=True argument to the function, to modify the original DataFrame.



Query DataFrame ¶

+ Code + Markdown

we are only interested in participants belonging to BP03 study and if they started after 2021-12-01

```
cutoff_date = pd.to_datetime("2021-12-01")

BP03_df = df.query('ISA == "BP03" and Data_collection_started > @cutoff_date')
```

```
BP03_df
```

	Site	SubjectID	ISA	Actual_Visit	Data_collection_started
7	102	12902	BP03	V3	2021-12-03
232	123	12899	BP03	V3	2021-12-10
272	142	12885	BP03	V3	2021-12-03

A logical expression that specifies which rows to return in the output

(this expression must be formatted as a string)

The name of your DataFrame

```
yourDataFrame.query(expression, inplace = False)
```

The name of the method

A True / False flag that indicates whether you want to operate directly on the DataFrame or create a new output DataFrame

```
     name region  sales  expenses
1    Emma  North  52000     43000
3  Markus  South  34000     44000
5  Thomas   West  72000     39000
7  Olivia   West  55000     60000
9   Anika   East  65000     44000
```

```
sales_data.query('sales < expenses')
```

```
     name region  sales  expenses
3  Markus  South  34000     44000
7  Olivia   West  55000     60000
```

We're going to compare two variables – sales and expenses – and return the rows where sales is less than expenses.

This is pretty straightforward. Our logical expression, 'sales < expenses', instructs the query method to return rows where sales is less than expenses. And that's what is in the output!

Keep in mind that we can use more than two variable in our expressions, or make the expressions more complicated with logical operators.

What's the advantage of using .query() over brackets?

```
sales_data[sales_data['sales'] > 50000]
```

```
sales_data.query('sales > 50000')
```

- ***Cleaner***
- ***Easier to read and write***
- ***More like "English"***
- ***More powerful when in conjunction with other methods***

Query in PANDAS compare to SQL CODE...

```
SELECT column1, column2, ...
FROM table_name
WHERE condition
```

If you're coming from an SQL background, you might be trying to figure out how to re-write your SQL queries with Pandas.

If that's the case, you need to know how to translate SQL syntax into Pandas.
So if you were trying to translate SQL into Pandas, how does .query() fit in?

The query method is like a where statement in SQL.

You can use query to specify conditions that your rows must meet in order to be returned.

Pandas DataFrame to Heatmap

Heatmap *is a graphical representation of data using different colors where the color represents values.*

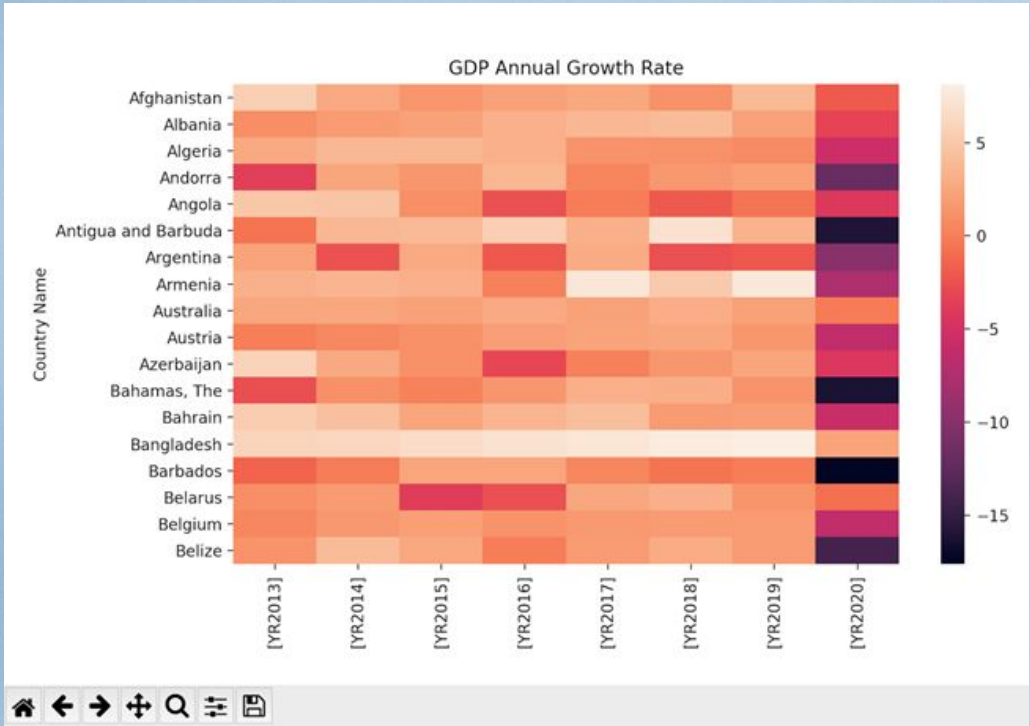
Most real estate, engineering, marketing, pharmaceutical, and research sectors use Heatmap for data analysis.

Heatmaps are the best tool for visualizing complex and simple information compared to charts or tables. For example, Businesses use Heatmap to visually analyze their sales, raw materials usage, and financial data.

Pandas DataFrame to Heatmap

When to use Heatmap (and benefits)?

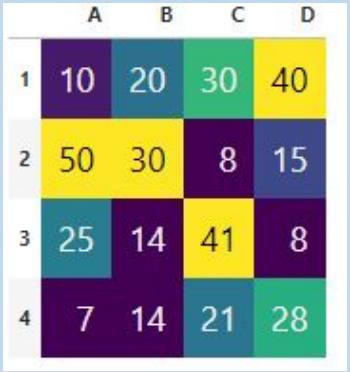
- **Enhance communication**
  - *Heatmap is a more effective tool to communicate the business's current financial or operational situation. And provide us with information for improvements to be made.*
- **Enhance Time based trend analysis**
  - *The most extraordinary feature about Heatmap can convey timely changes using visual representation. Organizations can see improvement or decline in their sales or other data over time and in which locations. It helps companies to decide on sales and marketing efforts accordingly.*
- **Enhance competitive advantage**
  - *Heatmaps can help us to study the competitive landscape of the market. Businesses can identify the scope to increase their sales in respective competitors' locations by using numerical data in heatmaps.*



Best practices:

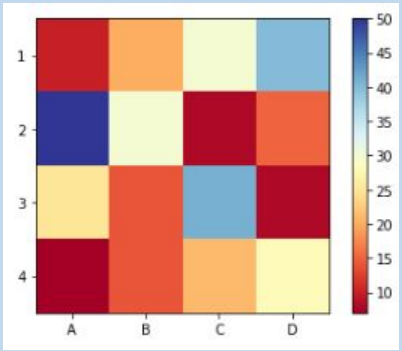
- Select the right color palette
  - The color is the primary element in this type of chart. So, it is crucial to select the correct color palette to match the data. Usually, the lighter color represents better results, and the darker color represents the worst case.
- Always include a legend
  - The general rule for any graph is to include a legend, and it provides us the reference details. Legend in the Heatmap is the color bar. The color bar shows the range of values with different densities of color.
- Shows values in cells
  - Displaying the values in each cell in the heat map is an excellent idea. It would be significantly easier to read each cell. Or else, we have to look at the color bar each time to see the value for the specific color.

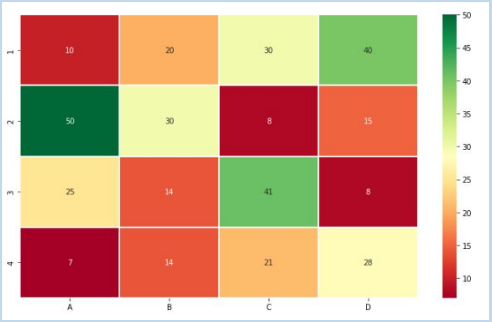
Heatmaps from different methods/libraries



pandas

matplotlib

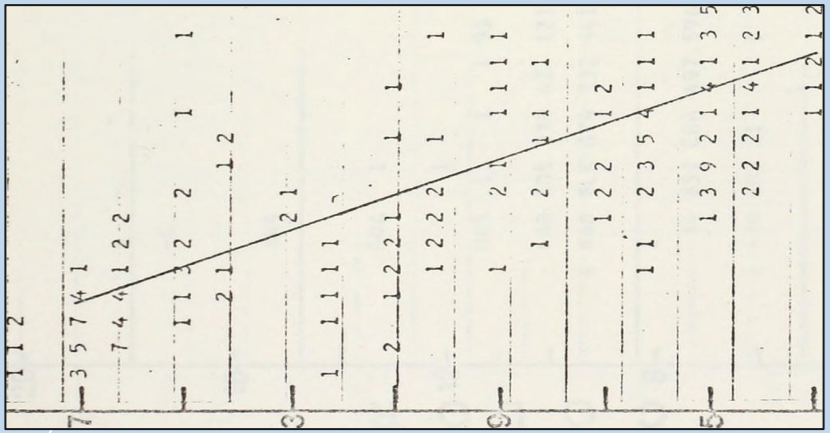




seaborn

Correlation analysis and visualization using Pandas

Correlation is a way to determine if two variables in a dataset are related in any way.



Statistics and data science are often concerned about the relationships between two or more variables (or features) of a dataset. Each data point in the dataset is an observation, and the features are the properties or attributes of those observations.

Correlation is a way to determine if two variables in a dataset are related in any way.
Correlations have many real-world applications.

We can see if using certain search terms are correlated to views on youtube. Or, we can see if ads are correlated to sales. When building machine learning models correlations are an important factor in determining features.
Not only can this help us to see which features are linear related, but if features are strongly correlated we can remove them to prevent duplicating information.

Measuring correlation

Rvalue

$$r = \frac{\sum(x-\bar{x})(y-\bar{y})}{\sqrt{\sum(x-\bar{x})^2 \sum(y-\bar{y})^2}}$$

In data science we can use the r value, also called Pearson's correlation coefficient. This measures how closely two sequences of numbers (i.e., columns, lists, series, etc.) are correlated.

The r value is a number between -1 and 1. It tells us whether two columns are positively correlated, not correlated, or negatively correlated. The closer to 1, the stronger the positive correlation. The closer to -1, the stronger the negative correlation (i.e., the more "opposite" the columns are). The closer to 0, the weaker the correlation.

We aren't going to explain the math behind the r value, but if you are curious, some youtube video does a great job. Instead, let's visualize correlations with a simple dataset.

EXAMPLE

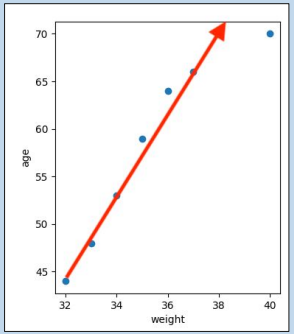
Weight	Age(months)	Baby Teeth	Eye Color
33	48	19	1
36	64	12	3
34	53	18	2
40	70	5	2
32	44	20	3
37	66	10	1
35	59	15	2

The dataset shows data on seven children. It has the following columns, weight, age(in months), amount of baby teeth, and eye color. The eye color column has been categorized where 1 = blue, 2 = green, and 3 = brown.

Let's make 3 scatter plots using the above data. We'll look at the following 3 relationships: age and weight, age and baby teeth, and age and eye color.

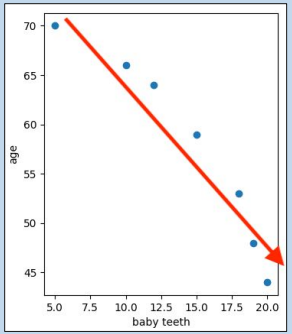
When we look at the correlation between age and weight the plot points start to form a positive slope. When we calculate the r value we get 0.954491. With the r value so close to 1, we can come to the conclusion that age and weight have a strong positive correlation. Intuitively, this should check out. In a growing child, as they get older and grow they start to weigh more.

Conversely, the plot points on the age and baby teeth scatter plot start to form a negative slope. The r value of this correlation is -0.958188. This signifies a strong negative correlation. Intuitively, this also makes sense. As a child gets older they lose their baby teeth.



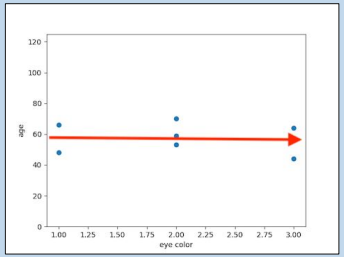
Age and Weight

Positive correlation



Age and Baby teeth

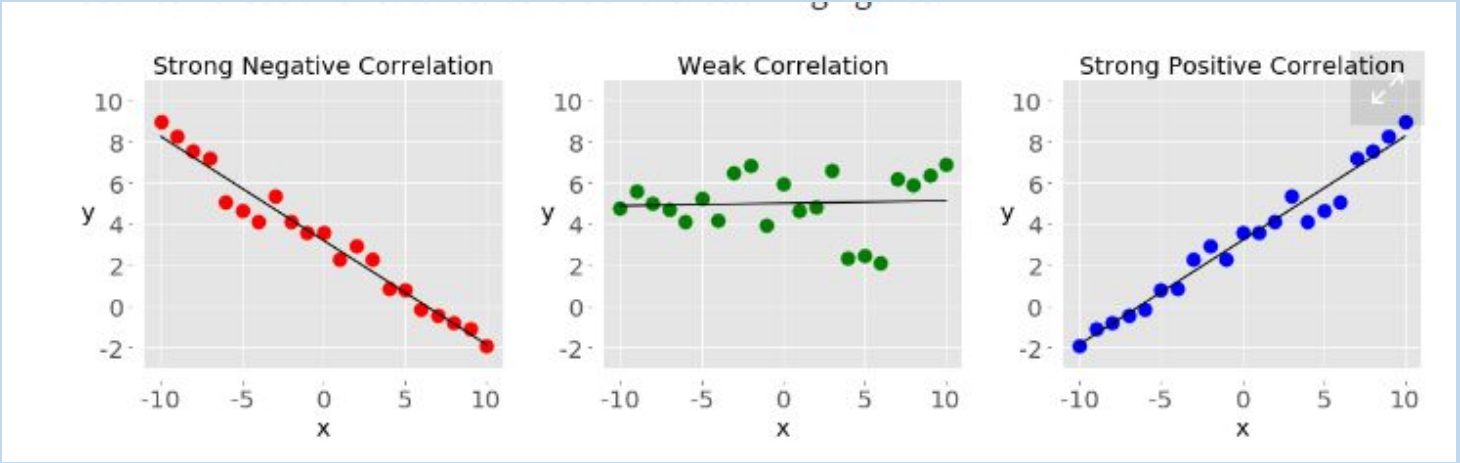
Negative correlation



Age and Eye Color

No correlation

On our last scatterplot, we see some plot points with no clear slope. This correlation has an r value of -0.126163. There is no significant correlation between age and eye color. This should also make sense as eye color shouldn't change as a child gets older. If this relationship showed a strong correlation we would want to examine the data to find out why.



Negative correlation (red dots): In the plot on the left, the y values tend to decrease as the x values increase. This shows strong negative correlation, which occurs when *large* values of one feature correspond to *small* values of the other, and vice versa.

Weak or no correlation (green dots): The plot in the middle shows no obvious trend. This is a form of weak correlation, which occurs when an association between two features is not obvious or is hardly observable.

Positive correlation (blue dots): In the plot on the right, the y values tend to increase as the x values increase. This illustrates strong positive correlation, which occurs when *large* values of one feature correspond to *large* values of the other, and vice versa.

```
Python
>>> import pandas as pd
>>> x = pd.Series(range(10, 20))
>>> x
0    10
1    11
2    12
3    13
4    14
5    15
6    16
7    17
8    18
9    19
dtype: int64
>>> y = pd.Series([2, 1, 4, 5, 8, 12, 18, 25, 96, 48])
>>> y
0     2
1     1
2     4
3     5
4     8
5    12
6    18
7    25
8    96
9    48
dtype: int64
>>> x.corr(y)                    # Pearson's r
0.7586402890911867
>>> y.corr(x)
0.7586402890911869
```

Two scenarios could lead to spurious correlations:

- First, it could happen that there might be a third, "confounding" variable that is causing the two variables. Imagine there is a strong positive correlation between ice cream consumption and the number of heatstrokes in a given population. One may come to the absurd conclusion that eating ice cream is causing heatstrokes. However, what happens is both variables depend on a third one, most likely, the temperature. The higher the temperature, the more people will love to have ice cream, and also the more people will be likely to have a heatstroke.
- A second reason to be cautious with correlation is that dependencies between variables could be simply due to chance. This happens all the time, but more likely when we deal with small samples.

Another reason to take correlation with caution is the world we live in is complex. A stronger correlation between two variables can easily lead us to conclude that one caused the other. The belief that a given effect has only one cause is called monocausality, but this is rarely the case. Several variables can be behind a certain effect. Thus, in order to consider causality, we first need to assume all the elements of a system and evaluate them not only quantitatively (i.e. through statistical methods) but also qualitatively.

Correlation does not imply causation!

The correlation only quantifies the strength and the direction of the relationship between two variables. There might be a strong correlation between two variables, but it does not allow us to conclude that one causes the other. When strong correlations are not causal, we call them spurious correlations.

Size of Correlation	*Interpretation*
.90 to 1.00 (-.90 to –1.00)	Very high positive (negative) correlation
.70 to .90 (-.70 to -.90)	High positive (negative) correlation
.50 to .70 (-.50 to -.70)	Moderate positive (negative) correlation
.30 to .50 (-.30 to -.50)	Low positive (negative) correlation
.00 to .30 (.00 to -.30)	Little if any correlation

A Guide to Pandas Pivot Table

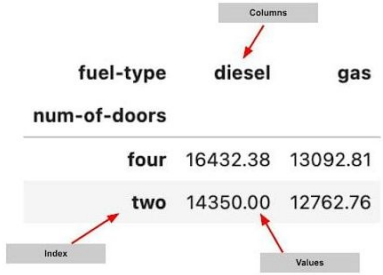
Pandas **pivot_table function** operates similar to a spreadsheet, making it easier to ***group***, ***summarize*** and ***analyze*** your data.

```
pivot = np.round(pd.pivot_table(data, values='price',
                                index='num-of-doors',
                                columns='fuel-type',
                                aggfunc=np.mean),2)
pivot
```

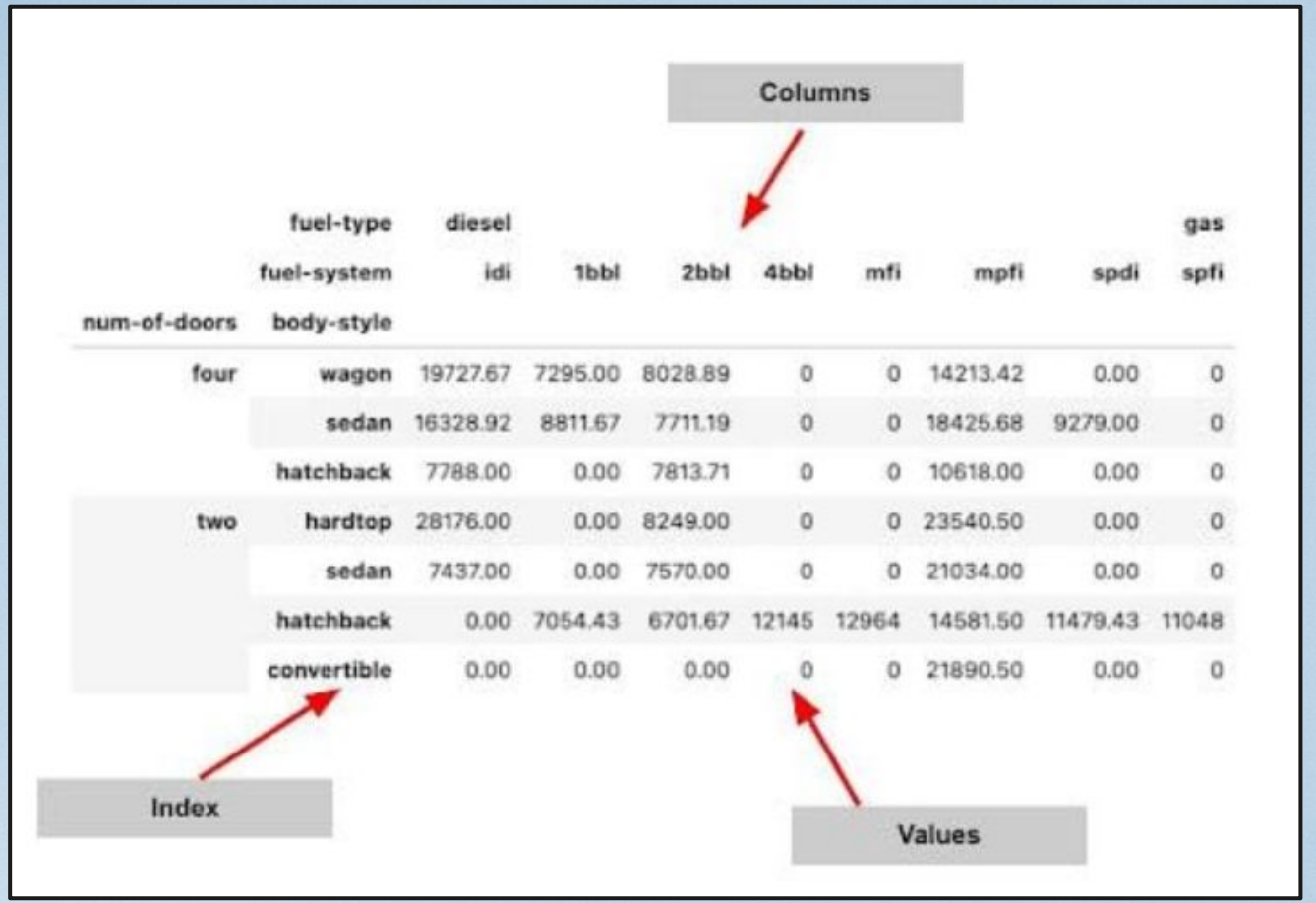
```
pivot = np.round(pd.pivot_table(data, values='price',
                                index=['num-of-doors', 'body-style'],
                                columns=['fuel-type', 'fuel-system'],
                                aggfunc=np.mean,
                                fill_value=0),2)
pivot
```

A pandas pivot table has three main elements:

- **Index**: This specifies the row-level grouping.
- **Column**: This specifies the column level grouping.
- **Values**: These are the numerical values you are looking to summarize.




```
np.round(pd.pivot_table(data, values='price',
                                index=['make'],
                                columns=['num-of-doors'],
                                aggfunc=np.mean,
                                fill_value=0),2).plot.barh(figsize=(10,7),
                                                           title='Mean car price by make and
number of doors')
```



	fuel-type	diesel							gas
	fuel-system	idi	1bbl	2bbl	4bbl	mfi	mpfi	spdi	spfi
num-of-doors	body-style								
four	wagon	19727.67	7295.00	8028.89	0	0	14213.42	0.00	0
	sedan	16328.92	8811.67	7711.19	0	0	18425.68	9279.00	0
	hatchback	7788.00	0.00	7813.71	0	0	10618.00	0.00	0
two	hardtop	28176.00	0.00	8249.00	0	0	23540.50	0.00	0
	sedan	7437.00	0.00	7570.00	0	0	21034.00	0.00	0
	hatchback	0.00	7054.43	6701.67	12145	12964	14581.50	11479.43	11048
	convertible	0.00	0.00	0.00	0	0	21890.50	0.00	0